Andromeda Spec 10: Reorder Buffer, Commit and CSRs

Oct 10, 2026 · @Jerry

Part of the Andromeda spec set. Shared types and conventions are defined in file 00.

Modules: `rob`, `csr_unit` · **Milestone:** Writeback and reorder buffer

1. What this unit does

The reorder buffer (ROB) is a 32-entry circular buffer that records every in-flight instruction in program order. Results arrive out of order on the 4 writeback buses; the ROB retires up to 4 complete instructions per cycle from its head, in order. Commit is the only point where architectural state changes, which makes exceptions and mispredicts precise. The ROB also decides every flush.

`csr_unit` holds the machine-mode control and status registers and takes traps.

2. How it works

2.1 Life of a ROB entry

1. **Allocate (dispatch, in order).** The instruction gets the entry at the tail, and the tail advances. The entry records what will change at commit: `rd_arch`, the new `prd`, and the previous mapping `old_prd`. The entry's index plus a wrap bit becomes the instruction's `rob_id`, which travels with it.
2. **Complete (writeback, out of order).** When a unit finishes, it reports on its writeback bus with the `rob_id`. The ROB marks the entry complete and stores any exception, branch outcome or FP flags. The result *data* goes to the physical register file, not the ROB.
3. **Commit (in order).** The ROB looks at the head. If it's complete, it retires: the rrat maps `rd_arch` to `prd`, and `old_prd` goes back to the free list. Up to 4 entries retire per cycle, stopping at the first that isn't done.

2.2 Why the ROB holds the mapping

The RAT only knows the newest mapping, and by the time an instruction commits, younger instructions have usually changed it. So the mapping each instruction needs at commit (`rd_arch`, `prd`, `old_prd`) is captured at rename and stored in its ROB entry. The `rob_id` is also why completion can be out of order: a unit doesn't need to know anything about program order, only which entry to mark.

2.3 Example

A slow `fdiv` is at the head, and three `add`s behind it finish first. The adds are marked complete but wait. Their results are already in physical registers, so dependents can use them. When the `fdiv` finishes, all four retire in the same cycle, in order.

2.4 Commit rules

Each cycle, scanning from the head:

1. Stop at the first entry that isn't complete.

2. An entry with an exception is **not** retired. The trap goes to `csr_unit`, fetch restarts at `mtvec`, and everything flushes.
3. A mispredicted branch retires, then flushes, and fetch restarts at its correct target.
4. A serializing instruction that changes how later instructions behave (CSR write, MRET, FENCE.I) retires, then flushes, and fetch restarts at PC + 4 (or `mepc` for MRET).
5. At most one control-flow instruction retires per cycle, because the predictor takes one training update per cycle.
6. Interrupts are taken between instructions: when one is pending and enabled, the head is not retired, and the core traps with `mepc` = the head's PC.

2.5 Exceptions are just flags until the head

A unit that detects a fault (illegal instruction, misaligned address, access fault, ECALL) doesn't stop anything. It reports the fault through `wb_t`, and it's recorded in the ROB entry. If the faulting instruction turns out to be on a mispredicted path, it is flushed before reaching the head and the fault never happens. Only a fault at the head is real, and at that point every older instruction has committed and no younger one has.

2.6 Taking a trap

1. The ROB raises `trap_valid` with the cause, PC and `tval`.
2. `csr_unit` writes `mepc`, `mcause` and `mtval`, and updates `mstatus` (MPIE takes MIE, MIE clears).
3. The ROB requests a flush from `pipe_ctrl` with `mtvec` as the restart PC.

MRET is the reverse: it retires, `csr_unit` restores MIE from MPIE, and the flush restarts fetch at `mepc`.

3. Interface

3.1 rob

Each entry holds: `pc`, `instr` (for the trace), `fu_type`, `rd_en`, `rd_fp`, `rd_arch`, `prd`, `old_prd`, `is_load`, `is_store`, branch type and `bp_meta`, `complete`, `exc_valid`, `exc_cause`, `exc_tval`, `fflags`, `br_taken`, `br_target`, `br_mispredict`, `serialize`.

Signal	Dir	Width	Description
`flush`	in	1	`be_flush` (the ROB empties itself after raising it)
`alloc_valid` / `alloc_uop`	in	4 / 4 × uop_t	New entries from dispatch
`alloc_id`	out	4 × 6	Tail + 0..3
`free_cnt`	out	6	Free entries
`head_id`	out	6	Oldest entry, for serializing uops and age checks
`wb`	in	4 × wb_t	Completion, exceptions, branch outcomes, fflags
`cm_valid` / `cm_rd_en` / `cm_rd_fp` / `cm_rd_arch` / `cm_prd` / `cm_old_prd`	out	4 × (1 / 1 / 1 / 5 / 7 / 7)	Retirement to rename (rrat, free lists)

Signal	Dir	Width	Description
cm_load_cnt / cm_store_cnt	out	3 / 3	Retired loads and stores, to lsu
fflags_accrue	out	5	OR of fflags of retired FP instructions
instret_inc	out	3	Instructions retired this cycle
trap_valid / trap_cause / trap_pc / trap_tval / trap_is_irq	out	1 / 5 / 32 / 32 / 1	Exception or interrupt being taken
mret_valid	out	1	MRET retired
irq_pending	in	1	From csr_unit
mtvec / mepc	in	32 / 32	Restart addresses from csr_unit
flush_valid / flush_pc	out	1 / 32	To pipe_ctrl
trace_*	out		Trace port fields listed in file 01

3.2 csr_unit

Holds the machine-mode CSRs: mstatus, misa, mie, mip, mtvec, mscratch, mepc, mcause, mtval, mhartid, mcycle(h), minstret(h), fflags, frm, fcsr. CSR instructions reach it from P1 only when they are at the ROB head, so they always see fully committed state.

Signal	Dir	Width	Description
req_valid / req_op / req_addr / req_wdata	in	1 / 3 / 12 / 32	CSRRW/S/C (and immediate forms) from alu_bru
resp_valid / resp_rdata / resp_illegal	out	1 / 32 / 1	Old value, or illegal CSR or access
trap_valid / trap_cause / trap_pc / trap_tval / trap_is_irq	in	1 / 5 / 32 / 32 / 1	Writes mepc, mcause, mtval and updates mstatus
mret_valid	in	1	Restores mstatus.MIE from MPIE
fflags_accrue	in	5	ORed into fflags
instret_inc	in	3	Added to minstret
irq_timer / irq_external	in	1 / 1	Set mip bits
irq_pending	out	1	Enabled interrupt pending
mtvec / mepc	out	32 / 32	To the ROB
frm	out	3	To fpu_top

4. Implementation notes

- **Split the storage by who writes it.** The fields written at dispatch (pc, registers, type) and the fields written at writeback (complete, exception, branch outcome, fflags) have different write ports. Store them in separate arrays: a payload array with 4 write ports indexed by the tail, and a small status array with 4 write ports indexed by `rob_id`. This keeps the write port muxes small.
- **Wrap bit.** Head and tail are 6 bits: a 5-bit index plus a wrap bit. Equal indices with equal wrap bits mean empty; equal indices with different wrap bits mean full. The same trick makes `rob_id` comparisons work for age.
- **Commit logic.** Read the 4 entries starting at the head, compute "complete and no exception" for each, and take the longest prefix, cut by rules 2 to 6 above. The find-first-N picker from file 00 fits this.
- **Flush timing.** The ROB decides a flush at commit and `pipe_ctrl` registers it. Make sure nothing retires in the cycle after a flush decision, and that the ROB clears to empty (head = tail) as part of the flush.
- **`pc` storage.** Storing a full 32-bit PC per entry is the simple choice and makes the trace port trivial. Real cores save area by storing one PC per fetch block; not worth it in v1.
- **CSR details.** `mcycle` increments every cycle, `minstret` by `instret_inc`. `mstatus.MPP` can be hardwired to machine mode since there are no other privilege modes. Check the privileged spec's rules for `mstatus.FS`, which exists because the F extension is implemented. `misa` should report RV32 with I and F. `fcsr` is a view of `frm` and `fflags` together.
- **WFI.** Can be implemented as a no-op in v1, which the spec allows.

5. Verification checklist

- ☐ Unit test with random allocation, out-of-order completion and commit against a software ROB model
- ☐ Commit stops at the first incomplete entry; never retires out of order
- ☐ Exception at the head: entry not retired, trap taken, flush to `mtvec`
- ☐ Exception on a wrong-path instruction never traps
- ☐ Mispredicted branch retires, then flushes to the correct target
- ☐ Only one branch retires per cycle
- ☐ Interrupt taken with `mepc` = head PC; MRET returns correctly
- ☐ `fflags` accrue only from committed FP instructions
- ☐ Head and tail across many wraparounds; full and empty detection
- ☐ CSR reads see committed state; writes to `frm` affect only later instructions